AGRO FINTECH

Capstone Project — Peer Review Assessment

Evidence-based review of the submitted project ZIP
Assessment date: 4 November 2026

Review scope	12 peer-review criteria covering tests, API, data pipeline, models, EDA, deployment and monitoring.
Evidence reviewed	main.py, prediction.py, train.py, data_cleaning.py, streamlit_app.py, README.md, requirements.txt, model artifacts and project directory contents.
Review method	Static inspection of the uploaded ZIP. No live database/API execution or deployment test was performed.

Overall assessment

The submission demonstrates a useful end-to-end prototype: data preparation, two Decision Tree prediction tasks, a FastAPI prediction endpoint, and a Streamlit front end. However, it is **not yet demonstrated as a production-ready or fully peer-review-compliant system**. The most important gaps are the absence of automated tests, performance monitoring, Docker configuration, visual EDA evidence, and a true comparison against alternative models or a baseline.

Important implementation concern: the submitted prediction.py loads artifacts from ./model/ (lowercase), while the ZIP contains a MODEL/ directory (uppercase). On case-sensitive systems, this path mismatch can prevent the API from loading its models unless the artifacts are placed in the expected lowercase directory.

Peer-review answers

1. Are there unit tests for the API?

NO No API test files or test suite were found in the submitted project. main.py defines a FastAPI app with a GET / route and a POST /predict route, but there are no tests shown for valid requests, invalid inputs, response schemas, status codes, or error handling.

Evidence: main.py; project file listing.

2. Are there unit tests for the model?

NO train.py trains and evaluates two Decision Tree models and prints classification reports and confusion matrices. However, no unit tests are provided to verify preprocessing, feature ordering, encoder behavior, prediction outputs, or model loading.

Evidence: train.py, prediction.py; no tests/ directory or test modules found.

3. Are there unit tests for the logging?

NO No dedicated logging implementation or logging tests were found. The scripts primarily use print() statements for progress and diagnostic output. There are no demonstrated assertions for log messages, log levels, handlers, or log-file behavior.

Evidence: main.py, prediction.py, train.py, data_cleaning.py.

4. Can all unit tests be run with a single script, and do all tests pass?

NO There is no visible test suite, test-runner configuration, or single test command/script. Therefore, test execution and pass status cannot be established. The README documents commands to run the pipeline, training, API and UI, but these are application commands rather than a unit-test runner.

Evidence: README.md and project structure.

5. Is there a mechanism to monitor performance?

NO No monitoring or instrumentation mechanism was found for request latency, throughput, error rates, resource usage, or model drift. The README claims inference is under 1 ms, but the submitted code does not include a benchmark or ongoing monitoring implementation to substantiate that claim.

Evidence: README.md, main.py, prediction.py.

6. Was there an attempt to isolate read/write unit tests from production models and logs?

NO No read/write unit tests or test fixtures were found, and no test-specific model/log directories or temporary storage strategy is demonstrated. The .gitignore excludes *.log files, but that alone does not isolate test artifacts from production artifacts.

Evidence: .gitignore; absence of tests and test configuration.

Peer-review answers (continued)

7. Does the API work as expected, including predictions for a specific country and all countries combined?

PARTIAL A FastAPI application and POST /predict endpoint are implemented for a single farmer input, returning Risk_Level and Loan_Approval. However, the request schema has no country field, and there are no country-specific or all-country aggregate prediction endpoints. In addition, prediction.py loads ./model/... while the supplied archive contains MODEL/... (uppercase), which can break model loading on case-sensitive operating systems. The API was not run against a live environment, so successful runtime behavior is not verified.

Evidence: main.py, prediction.py, README.md, MODEL/ directory.

8. Does data ingestion exist as a function or script to facilitate automation?

PARTIAL data_cleaning.py provides a reusable script-like pipeline that reads PostgreSQL tables, aggregates and merges data, handles missing values, engineers features, and writes a CSV. This is a meaningful basis for automation. However, the workflow executes at module level rather than being encapsulated in functions or a main guard, and database credentials/configuration are hard-coded. It is therefore not yet a robust, parameterized ingestion component.

Evidence: data_cleaning.py; README.md pipeline instructions.

9. Were multiple models compared?

PARTIAL Two Decision Tree models are trained: one predicts risk level and another predicts loan approval. These are separate prediction tasks, not a comparison of competing algorithms for the same target. No Random Forest, Logistic Regression, Gradient Boosting, or other alternative is evaluated side-by-side with common metrics.

Evidence: train.py; README.md Model Architecture and Training sections.

10. Did the EDA investigation use visualizations?

NO The visible data-preparation script reports shapes, sample rows, missing-value counts and columns using print statements. No EDA charts, plotting code, or notebook containing visual exploration was found in the archive.

Evidence: data_cleaning.py; submitted project file listing.

11. Is everything containerized within a working Docker image?

NO No Dockerfile, docker-compose.yml, or other container build configuration was found. The README itself lists Dockerfile and Docker Compose as future infrastructure improvements, indicating containerization is planned rather than included. A working Docker image was not supplied.

Evidence: README.md Extending the System section; project file listing.

12. Did they use a visualization to compare their model to the baseline model?

NO The training script prints confusion matrices and classification reports, but it does not generate a visualization comparing a trained model with a baseline. No baseline model or baseline comparison chart was found.

Evidence: train.py; README.md Training section.

Summary scorecard

#	Criterion	Assessment
1	1. Are there unit tests for the API	NO
2	2. Are there unit tests for the model	NO
3	3. Are there unit tests for the logging	NO
4	4. Can all unit tests be run with a single script, and do all tests pass	NO
5	5. Is there a mechanism to monitor performance	NO
6	6. Was there an attempt to isolate read/write unit tests from production models...	NO
7	7. Does the API work as expected, including predictions for a specific country ...	PARTIAL
8	8. Does data ingestion exist as a function or script to facilitate automation	PARTIAL
9	9. Were multiple models compared	PARTIAL
10	10. Did the EDA investigation use visualizations	NO
11	11. Is everything containerized within a working Docker image	NO
12	12. Did they use a visualization to compare their model to the baseline model	NO

Counts

Yes: 0 Partial: 3 No: 9

This is an evidence-coverage score based on the submitted ZIP, not a judgement that the underlying ideas or code can never work. Items marked No mean the required evidence or implementation was not present in the archive.

Recommended improvements before peer review

1. Add automated tests — Create pytest tests for API routes, model inference, preprocessing/encoders, invalid inputs and logging. Add fixtures with small synthetic inputs; do not write to production model or log locations.

2. Add one test command and CI — Configure pytest and provide a single command such as pytest -q. Add a CI workflow so tests run automatically on each push or pull request.

3. Fix model artifact paths — Make the artifact directory name consistent (model vs MODEL) and resolve paths relative to the Python file or through configuration. Add a startup check that reports missing artifacts clearly.

4. Add country-level capability if required — Retain country/state identifiers where needed and implement explicit single-country and all-country prediction/aggregation endpoints. Add API tests for both cases.

5. Make ingestion automation-ready — Move pipeline steps into functions, add a main guard, read database settings from environment variables, and provide clear failure handling and repeatable output paths.

6. Compare models and baselines — Evaluate at least one simple baseline and multiple candidate algorithms on the same train/test split. Compare suitable metrics (for example precision, recall, F1 and ROC-AUC where appropriate) and visualize the results.

7. Add EDA visuals — Include plots for target/class distribution, missing values, numeric distributions, correlations and relevant geographic or crop-level patterns.

8. Add monitoring and Docker — Instrument API latency, request counts and error rates; define a Dockerfile and docker-compose setup for the API, UI and required services. Verify the image with a clean build and smoke test.

Final conclusion

Agro Fintech demonstrates a credible prototype with data preparation, supervised learning, model serialization, a prediction API and a user interface. Based on the submitted archive, it does not yet satisfy several engineering and evaluation expectations in the peer-review checklist. Prioritize automated tests, correcting and validating model loading, country-level API behavior (if required by the rubric), baseline/model comparisons, EDA visualizations, monitoring and Docker packaging.

Limitations of this review

This assessment is based on static inspection of the ZIP contents. The PostgreSQL pipeline was not executed because it depends on a configured database. The API was not launched, no HTTP prediction was submitted, and no Docker build or performance benchmark was run. Consequently, runtime success, numerical model quality, and deployment behavior remain unverified.